

Mango Tree — Legal Evidence Engine

Complete Developer Documentation

A court-ready, AI-powered fraud-investigation platform built on an email + document corpus.

This document is a full, code-grounded technical reference for the entire system: every ingestion path, the force-vision OCR cascade, chunking / contextual-retrieval / embedding, the knowledge graph and entity resolution, the money graph, the fraud detectors, the complete RAG v2 retrieval stack, the RAG v3 agentic investigator, the citation verifier, the backend API, the React frontend, the database layer, and the configuration surface. Every technique described here is traceable to actual source files, functions, constants, and models in the codebase.

Table of Contents

1. System Overview & Architecture
 2. Technology Stack
 3. Ingestion Subsystem (PST, EML/mbox, Gmail pull, real-time push)
 4. Deduplication & Evidence Spine
 5. Document Extraction & Force-Vision OCR
 6. Cleaning, Chunking, Contextual Summarization & Embedding
 7. Knowledge Graph & Entity Resolution
 8. Money Graph
 9. Fraud Detectors, Findings, Timeline & Grounded Facts
 10. Retrieval — RAG v2 (every technique)
 11. Agentic Investigator — RAG v3
 12. Verification, Provenance & Evidence Schema
 13. Backend API & Server
 14. Frontend Application
 15. Database Layer & Collection Catalog
 16. Configuration Reference
 17. Real-Time Automation & Deployment
 18. Appendix A — Constants & Tunables Quick Reference
 19. Appendix B — Known Configuration-Drift Notes
-

1. System Overview & Architecture

The Mango Tree Legal Evidence Engine is a Retrieval-Augmented Generation (RAG) platform purpose-built for **legal fraud investigation**. It ingests an entire mailbox (emails plus every attachment), transcribes every document with frontier vision OCR, builds a deduplicated vector-searchable corpus enriched with a knowledge graph of people / LLCs / properties / cases, and answers investigative questions through an **agentic Claude planner** that must cite verbatim, machine-verified evidence for every claim.

1.1 End-to-end data flow



1.2 Design principles baked into the code

- **Nothing is trusted without verbatim evidence.** Every fact in an answer carries a `verbatim_quote` and a `source_chunk_id`; a deterministic verifier (`src/rag/v2/verifier.py`) checks the quote actually appears in the cited chunk, using OCR-tolerant normalization and critical-token gating (money/date/percent must match, not just fuzzy-pass).
- **Force-vision OCR.** Born-digital text layers are deliberately bypassed; every page is transcribed by a vision model so scanned and digital documents are treated identically and consistently (`scripts/ocr_attachments_v2.py`, `src/extractor/*`).
- **Deduplicate content, fan-out provenance.** A byte-identical attachment appearing in 600 emails is chunked/embedded **once** (keyed by `sha256`), and every email that carried it is recorded in the chunk's `occurrences[]` array (Option B, `scripts/build_email_chunks_v2.py`).
- **Privilege is a first-class, fail-closed property.** "Clean mode" excludes privileged chunks at the vector-index layer so shareable answers cannot leak privileged strategy (`src/rag/provenance.py`, Atlas filter paths).
- **Idempotent, resumable pipelines.** Every heavy stage (OCR, chunk/embed, enrichment) skips already-done work by `sha256` / `email_id`, so runs can be re-executed safely.

1.3 The two application surfaces

1. **Investigate (Chat):** a streaming WebSocket chat where the agentic investigator answers questions with inline, color-coded citations, an expandable reasoning trace, an evidence drawer, and one-click PDF export.

2. **Investigation Workspace (REST):** instant, pre-materialized dashboards — Portfolio grid (with per-property fact counts and an ad-hoc LLM column), Property Detail (title chain, encumbrances, ownership timeline, flow of funds, findings, document viewer), Findings review, and a Daily Brief "insight engine."

2. Technology Stack

Layer	Technology	Where
Language / runtime	Python 3.9 (backend), Node/Vite (frontend)	repo-wide
Web server	FastAPI + Uvicorn + WebSockets	server.py, api/
Database	MongoDB Atlas (with Atlas Vector Search)	src/db/mongo.py
Binary storage	GridFS bucket attachment_files	src/db/repository.py
Embeddings	Voyage AI voyage-4-large (1024-dim, cosine)	src/rag/embedder.py
Reranker	Voyage rerank-2.5	src/rag/reranker.py
Generation / reasoning	Anthropic Claude — claude-opus-4-8 (agent), claude-sonnet-4-6 (rewrite/summary/OCR)	src/rag/v3, src/rag/v2
Vision OCR	Claude Sonnet 4.6 Vision → GPT-5 (OpenAI) Vision → RapidOCR (ONNX)	src/extractor/*
Token counting	tiktoken cl100k_base	src/rag/tokens.py
PDF/doc parsing	PyMuPDF (fitz), Pillow, python-docx, RapidOCR	src/extractor/*
Real-time	Gmail users.watch() + Google Cloud Pub/Sub	scripts/gmail_*, deploy/
Frontend	React 19, Vite, Ant Design, react-router, axios, react-markdown, jsPDF	frontend/src/
Auth	JWT (python-jose) + bcrypt (passlib)	api/auth.py

3. Ingestion Subsystem

The system supports **four ingestion paths**, all funneling into the same canonical `emails` + `attachments` (+ GridFS) storage with identical 3-way deduplication.

3.1 Path A — Batch PST ingestion

The original bulk-load path (`IngestionPipeline` + `_build_email_doc`) parses a Microsoft Outlook `.pst` export (via `libratom`), walks folders, extracts each message, and bulk-upserts through `EmailRepository.upsert_emails` (keyed by `pst_entry_id`). Attachments are stored to GridFS via `store_attachment` and linked with `link_attachments_to_email`.

Note (documented gap): the PST builder `_build_email_doc` does **not** stamp the evidentiary spine (`corpus / privilege_status / custody`) at ingestion — those are applied downstream by `corpus_for()` fallbacks and `scripts/tag_chunk_corpus.py`. The `.eml` and Gmail paths stamp it inline.

3.2 Path B — `.eml` / mbox folder ingestion

`scripts/ingest_email_folder.py` ingests a directory of raw `.eml` files (and `mbox_extract.py` is a standalone pre-processor that explodes an `mbox` into individual messages). Each message runs through the shared canonical parser and `ingest_one_email`, stamping `corpus/privilege` inline.

3.3 Path C — Gmail API pull (`scripts/ingest_gmail.py`)

A read-only Gmail ingestion CLI with three subcommands:

- `profile` — verifies the authenticated mailbox and message count.
- `labels` — lists every Gmail label ("folder") with counts, so exact label names can be supplied.
- `pull` — pulls messages for one or more labels + optional `--after/--before` date range (or a targeted `--ids-csv` of exact message ids), running each through the idempotent `ingest_one_email`. **Dry-run by default**; `--live` performs writes. The dry-run samples messages and prints a "cleanliness check" (how many real attachments would be kept vs. signature logos dropped).

Auth uses an OAuth client-secret JSON (`GMAIL_CLIENT_SECRET`, default `client_secret.json`) with a cached token (`GMAIL_TOKEN_PATH`, default `gmail_token.json`); scope is `gmail.readonly` (cannot modify the mailbox). The Gmail wrapper is `src/ingest/gmail_client.py` (`GmailClient`): `authenticate`, `get_profile`, `list_labels`, `resolve_labels`, `iter_message_ids` (label/query/date filtered), `get_raw`, `get_metadata`, `get_attachment`, `get_headers`, `get_full_summary` (full MIME part list without downloading bytes), plus the `watch/history` methods (§3.5).

3.4 Path D — Real-time Pub/Sub push (event-driven)

The production auto-ingest path. Architecture:

1. **Arming the watch** (`scripts/gmail_watch.py`): calls `Gmail.users.watch()` pointed at a Cloud Pub/Sub topic, scoped to the "Boris Lawsuit" label; persists `label_id`, `armed_history_id`, `last_history_id`, `topic`, and `watch_expiration` in the `gmail_watch_state` collection. Gmail watches expire ~7 days, so a systemd timer re-arms daily.
2. **The worker** (`scripts/gmail_push_worker.py`): an always-on Cloud Pub/Sub **pull** subscriber (`max_messages=1`, so heavy pipelines never overlap). On each notification it reads the new `historyId`, calls `client.list_history(start_history_id=...)` to resolve which message ids changed (union of `messagesAdded` and `labelsAdded` that carry the watched label), and invokes `process_gmail_ids(...)`. If history has expired (>~1 week), it falls back to a 2-day label scan. It advances `last_history_id` and `acks` only on success; a failure `nacks` (redelivery).
3. **The scoped pipeline** (`src/ingest/realtime_ingest.py`, `process_gmail_ids`): for exactly the new message ids, it (1) fetches + stores with 3-way dedup, (2) runs force-vision OCR scoped to new SHAs, (3) chunks + contextually summarizes + embeds, (4) enriches (`corpus` tag + entity backfill), (5) verifies parity. Crucially it runs a **scoped** `_sync_occurrences` instead of the ~18-minute global "Phase D" occurrence sync used by the batch builder, so a single new email finishes in seconds.

Operational caveat (root-caused in production): the real-time worker was deployed on a slim 1 GB server built from `requirements_server.txt`, which deliberately excludes the OCR/PDF libraries (`fitz`/`PyMuPDF`, `Pillow`, `RapidOCR`). Its first post-capture step (`ocr_attachments_v2.py --force-vision`) therefore crashes on import, aborting enrichment while capture still succeeds. Result: emails are captured but not chunked until a workstation backfill runs. The permanent fix is either a capture-only worker + workstation sweeper, or installing the OCR stack on a larger droplet.

3.5 Watch → history → message-id resolution

`GmailClient.watch(topic_name, label_ids, label_filter_action)` arms push; `stop_watch()` disables it;

`list_history(start_history_id, label_id)` returns the incremental change log.

`gmail_push_worker._message_ids_from_history` collects ids from both `messagesAdded` and `labelsAdded` entries whose `labelIds` include the watched label.

3.6 Run & error tracking

Every ingestion run is bracketed by `EmailRepository.start_run` / `finish_run` writing to `ingestion_runs` (origin, labels, date range, corpus, totals, status). Per-message failures are logged to `ingestion_errors` via `repo.log_error(run_id, key, stage, message)`.

4. Deduplication & Evidence Spine

4.1 Three-way deduplication

`ingest_one_email` (in `src/ingest/gmail_ingest.py`) resolves whether an incoming message already exists using **three independent keys**, checked in order:

1. `pst_entry_id` — the source-native id. For Gmail, `"gmail:" + message_id`.
2. `internet_message_id` — the RFC 5322 Message-ID header (stable across export formats). This is why the same email ingested from a PST export and from Gmail deduplicates correctly even though the source ids differ.
3. `content_hash` — a SHA-256 over the tuple `(from, to, subject_normalized, date, body[:5000])`, catching near-identical messages lacking a Message-ID.

If a match is found via message-id/content-hash but the incoming copy carries a new Gmail id, provenance is recorded (`_record_gmail_provenance`) without inserting a duplicate.

Attachment binaries are separately deduplicated by `sha256` of their bytes — the anchor for OCR-once and chunk-once behavior downstream.

4.2 "Real attachment" vs signature logo

The parser distinguishes genuine document attachments from inline signature graphics via `_is_signature_logo`: inline images (with a Content-ID) and small images (below a ~50 KB threshold) are dropped, so email-signature PNG/JPG logos never pollute the corpus. Only "real" attachments are stored and OCR'd. This filter is applied identically at ingest and mirrored in the Gmail dry-run cleanliness report.

4.3 The evidentiary spine (`src/rag/evidence_schema.py`)

Every chunk carries a legal-custody spine so answers are court-admissible:

- **Corpora** (`CORPORA`): `legal_correspondence`, `fraud_communications`, `property_records`, `insurance_records`, `corporate_records`, `court_records`, `financial_records`.
- **Privilege classes** (`PRIVILEGE_STATUSES`): `privileged`, `adverse_party`, `third_party`, `public_record`, `not_privileged`. Clean mode excludes `privileged`.
- **Evidentiary classes** (`EVIDENTIARY_CLASSES`): `party_admission`, `recorded_instrument`, `third_party_business_record`, `court_record`, `privileged_work_product`, `correspondence`.
- `CORPUS_DEFAULTS` maps each corpus to default `{privilege_status, evidentiary_class}` (e.g., `legal_correspondence` → `privileged` / `work_product`; `fraud_communications` → `adverse_party` / `party_admission`; `property_records` → `public_record` / `recorded_instrument`).
- `evidentiary_fields(...)` stamps FRE 901/902-style custody: `matter_id` (default `matter_001`), `corpus`, `privilege_status`, `evidentiary_class`, and a `custody` sub-doc `{custodian, source_file, sha256, ingest_run_id}`, plus a `bates_range`.

5. Document Extraction & Force-Vision OCR

The extraction subsystem (`src/extractor/` + `scripts/ocr_attachments_v2.py`) transcribes every attachment into `attachments_v2`, deliberately preferring vision transcription over any born-digital text layer.

5.1 The force-vision policy

Force-vision is a **sentinel trick, not a flag**: `ocr_attachments_v2.py` sets `ocr_min_chars = 10_000_000` when `--force-vision` is passed (`src/extractor/...`, called from `_ocr_one_sha256`). Because no page can ever contain 10 million characters of extractable born-digital text, **every page is classified as "needs OCR"** and routed to the vision cascade. This makes scanned and digital PDFs behave identically and avoids trusting flaky embedded text layers. The batch/real-time pipelines pass `--force-vision`.

5.2 The OCR model cascade

For each page that needs OCR:

1. **Claude Sonnet 4.6 Vision** (`claude-sonnet-4-6`) — primary transcription (`src/extractor/claude_ocr.py`).
2. **GPT-5 Vision (OpenAI)** — fallback, triggered when Claude returns a `content_filter_block` (or otherwise fails) on a page. Recovery is per-page and logged ("page N: recovered via OpenAI vision").
3. **RapidOCR (ONNX, PP-OCR v4)** — final deterministic fallback when both vision models fail; capped at a maximum number of RapidOCR pages per document (12).

This cascade is why legal documents with redactions/stamps (which sometimes trip content filters) still get transcribed.

5.3 `extract_from_bytes` routing (`src/extractor/extractor.py`)

The entry point routes by file type: PDFs render page-by-page with **PyMuPDF (fitz)** at a configurable DPI (`OCR_VISION_DPI`, default 180 for vision; `OCR_DPI` 300 otherwise) and feed page images to the cascade; images go straight to vision; `.docx` is extracted natively (no vision needed — e.g., a 43 KB reply-draft extracted ~21K chars at \$0.00); `.html` is text-extracted (empty HTML parts are skipped with `skipped_reason=html_empty`); unsupported types (e.g., `.emz` Windows metafiles) are skipped via the rescue module with `rescue_unsupported_ext`.

It returns an `ExtractionResult` dataclass with: `method`, `char_count`, `avg_ocr_confidence`, `pages[]` (each `{page_no, method, ocr_confidence, char_count, text}`), `text` (concatenation), and `skipped_reason`.

5.4 Claude vision internals

- **Rate limiter**: `rpm=3000`, `in_tpm=1,500,000`, `out_tpm=300,000`, `max_tokens_per_call=1500` (armed on first use).
- **Spend guard**: `_SpendGuard` enforces a hard budget (`OCR_VISION_BUDGET_USD`, default \$15) across the whole run; every page's cost is metered.
- **Concurrency**: a `_MAX_INFLIGHT = 3` semaphore plus `OCR_VISION_MAX_CONCURRENCY` (default 8) bound parallel calls.
- **Image handling**: pages are rendered to PNG, base64-encoded, and sent with a transcription prompt; output is capped by `max_tokens`.

5.5 `attachments_v2` output schema

One row per **source attachment** `_id` (the same ObjectId as the legacy `attachments` row, so all foreign keys map without a translation table), deduped so each unique `sha256` is OCR'd once:

- `_id`, `email_id`, `sha256`, `filename`, `gridfs_id`, `size_bytes`
- `extracted_text` (full concatenated transcript)
- extraction sub-doc: `method`, `char_count`, `avg_ocr_confidence`, `page_count`, `pages[]` (per-page `{page_no, method, ocr_confidence, char_count, text}`), `extracted_at`, `skipped_reason`, `elapsed_sec`
- `extracted_via` ("vision_v2"), `extracted_at`

`ocr_attachments_v2.py` groups the legacy `attachments` collection by `sha256`, skips SHAs already in `attachments_v2` (resume), applies optional size filters and the new `--sha-file` scope (added for per-case OCR), and inserts with `ordered=False` so re-runs tolerate duplicates.

5.6 OCR confidence & quality gates

`scripts/stamp_ocr_confidence.py` copies each document's `ocr_confidence` and dominant page method onto its chunks and flags `ocr_low_confidence` when `conf < 0.6`. `scripts/repair_ocr_pages.py` re-runs a GPT-5 vision repair pass on low-confidence pages. The verifier downstream is OCR-tolerant (see §12), so minor OCR jitter does not break citation matching.

6. Cleaning, Chunking, Contextual Summarization & Embedding

This subsystem (`scripts/build_email_chunks_v2.py` + `src/rag/chunker.py` + `src/rag/tokens.py` + `src/rag/v2/contextual_summary.py` + `src/rag/embedder.py`) turns the deduplicated corpus into the vector-searchable `email_chunks_v2` collection.

6.1 Email body cleaning

`src/cleaner/text_cleaner.py` (`clean_email_body`) applies encoding fixes (mojibake repair), quoted-reply and signature stripping, and whitespace normalization before chunking, so embeddings capture the new content of each message rather than the repeated quoted thread.

6.2 Chunking algorithm (`src/rag/chunker.py`)

Token budgets: `CHUNK_SIZE_TOKENS = 1000`, `CHUNK_OVERLAP_TOKENS = 200` (from `config/settings.py`). Token math uses `tiktoken cl100k_base` (a close proxy for Voyage's tokenizer).

- **Core splitter** `_chunk_text`: greedy paragraph-first (`\n{2,}`), then sentence-level split for oversized paragraphs, then a hard token-window slice for oversized sentences. Chunks are packed until the budget would overflow, then the next chunk is seeded with the last `overlap_tokens` tokens (sliding overlap).
- **Email bodies** (`chunk_email_body`): a one-line header `[Email – <date> | from <x> | to <y> | subject: <z>]` is built and its tokens reserved from the budget, then prepended to every chunk. `page_start/page_end` are `None`.
- **Attachments** (`chunk_attachment`): page-aware. Small pages accumulate into a buffer that flushes into one chunk (spanning multiple `page_nos`); large pages flush the buffer then split internally with overlap, tagging each sub-chunk with its page. Each chunk gets a header `[Attachment – <filename> | <date> | parent email: <subject> | p. N]` (or `pp. N-M`).

The `Chunk` dataclass carries both `text` (with header, used for embedding) and `body` (raw, used for highlighting), plus `n_tokens`, `chunk_index`, `page_start/end`.

6.3 Contextual retrieval (`src/rag/v2/contextual_summary.py`)

Implements Anthropic's **Contextual Retrieval**: before embedding, each chunk is prefixed with a 100–150 token LLM-written summary situating it in the whole document (doc type, date/period, parties/entities/addresses, what the chunk is about, and how it relates to the whole). This is documented to lift recall 35–50% on legal corpora.

- **Model**: `claude-sonnet-4-6`; client configured `timeout=90s`, SDK retries disabled in favor of explicit tenacity (`stop_after_attempt(4)`, exponential backoff).
- **Prompt caching (cost control)**: a document's chunks are processed sequentially; the whole document is sent once as a `cache_control: {"type": "ephemeral"}` block, so the first call writes the cache (~1.25× input) and subsequent chunks read it at ~0.1×. Caching engages when the doc is $\geq$ `_CACHE_MIN_TOKENS = 1024` and has ≥ 2 chunks; docs are trimmed to `_MAX_DOC_TOKENS_FOR_CONTEXT = 150_000`; output capped at `_MAX_CONTEXT_OUTPUT_TOKENS = 200`.
- **Fail-safe**: a per-chunk failure yields an empty context (the chunk is still embedded); the build never crashes.
- **Cost accounting**: `_Usage` tracks input/output/cache-write/cache-read tokens with Sonnet-4 rates; the build logs total cost (typical incremental runs cost cents).

6.4 Embeddings (`src/rag/embedder.py`)

- **Model**: Voyage `voyage-4-large`, **1024 dimensions**, cosine similarity. `input_type="document"` for corpus, `input_type="query"` for queries; API-side `truncation=True`.
- **Rate limiting**: a sliding 60-second window limiter enforcing both RPM and TPM (defaults target the free tier 3 rpm / 10k tpm; with a card, 2000 rpm / 1M tpm via `VOYAGE_RPM` / `VOYAGE_TPM`).
- **Batching**: greedy batches capped at 128 texts and ~9000 tokens/request; tenacity retry (`stop_after_attempt(6)`, exponential backoff). The `_Flusher` writes in batches of 64.

6.5 The v2 chunk schema (email_chunks_v2)

Written by the build:

Field	Meaning
source_type	"attachment" or "email_body" (primary discriminator)
sha256	file content hash (attachments); synthetic email:<id> for bodies
chunk_index, total_chunks	position within the document
text	context-prefix + header + chunk text (this is what is embedded)
body	raw chunk text (for highlighting)
context	the Claude-generated situating summary
n_tokens	token count of text
embedding, embedding_model	1024-d Voyage vector + "voyage-4-large"
page_start, page_end	page span (attachments)
extension, filename	file metadata (mirror of primary occurrence)
occurrences[]	every email carrying this content: {email_id, attachment_id, filename, date, date_ym, from_email, to_emails, subject, folder_path}
latest_date	max(occurrences[].date)
email_id, attachment_id, date, date_ym, from_email, to_emails, subject, folder_path	mirror fields of the primary (earliest) occurrence
created_at	flush timestamp

Stamped by downstream enrichment (same collection):

- corpus, privilege_status, privilege_basis, evidentiary_class — scripts/tag_chunk_corpus.py
- doc_authority_score, doc_source_type — scripts/stamp_authority.py
- entity_ids, entity_refs.{people,llcs,orgs,properties,cases}, primary_property_id, touches_david, entity_sides — scripts/backfill_chunk_entities.py
- ocr_confidence, ocr_method, ocr_low_confidence — scripts/stamp_ocr_confidence.py

Indexes (_ensure_v2_indexes) include sha256+chunk_index, source_type, latest_date, date, date_ym, from_email, filename, email_id, and occurrence-fanout indexes (occurrences.email_id/from/date/filename). The Atlas vector index (email_chunks_v2_vector) and BM25 text index are created separately.

6.6 Occurrences fan-out (Option B) — Phases A–D

- **Phase A** (_gather_jobs): one pass over all emails builds sha256 → [occurrences] for attachments and a list of body email-ids; skips attachments with no sha / no text / missing from attachments_v2.
- **Primary + mirror fields**: occurrences are sorted earliest-first (_date_sort_key); occurrences[0] is the primary whose metadata is mirrored to top-level fields so BM25/sort/Atlas filters work without unwinding arrays.
- **Phases B & C**: attachments and bodies are chunked, contextually summarized (thread pool, --workers default 16), embedded and written via the _Flusher.
- **Phase D (occurrence sync)**: for already-chunked SHAs that gained new parent emails, it reconciles occurrences[] and mirror fields **without re-embedding or calling Claude**. This is the ~17-minute global step the real-time pipeline replaces with a scoped _sync_occurrences.

6.7 Idempotency / resume

Under normal runs the build pulls the set of already-chunked SHAs and email-ids via aggregation and removes them from the todo lists, so only new content is processed. Per-document writes are all-or-nothing (delete_many then insert_many(ordered=False)), so partial writes are re-done, never duplicated. --force re-embeds; --no-embed is a dry chunk/context pass.

7. Knowledge Graph & Entity Resolution

The graph subsystem (`src/graph/`) models the real-world actors and assets behind the corpus and links them to evidence.

7.1 Entity model (`src/graph/schema.py`, `normalize.py`)

- **Kinds** (`ENTITY_KINDS`): `person`, `llc`, `org`, `property`, `case`.
- **Sides** (`SIDES`): e.g. `david_network`, `third_party`, `co_victim`, `our_side`; each entity carries a `side` and an `is_david` flag marking the adverse network.
- **Aliases & addresses**: entities have alias lists and (for properties) addresses; `normalize.py` provides `addr_core` (canonical address key), `llc_matches_address`, `norm_name`, `strip_suffixes`.
- **Edge types** (`EDGE_TYPES`), **authority scores** (`AUTHORITY_SCORES`), and **date kinds** (`DATE_KINDS`) are also defined here.

7.2 Entity resolution & merging

- **Resolution** (`resolve.py`, `scripts/resolve_entities.py`): a 3-tier `resolve_component` — exact key match → fuzzy match (≥ 92) → create-new.
- **Conservative auto-merge** (`scripts/merge_duplicate_entities.py`): merges only on an exact suffix-stripped key **with a side firewall** (never merges across adverse/victim sides). Fuzzy matches in the grey zone `[88,100)` are written to an `entity_review` collection for human adjudication.
- **Grey-zone LLM judge** (`scripts/resolve_grey_zone.py`): an LLM cross-encoder decides ambiguous pairs.
- **Human review loop** (`scripts/apply_entity_review.py`, `apply_entity_sides.py`): applies reviewed decisions and learns aliases, and assigns sides.

7.3 Chunk ↔ entity linkage (`scripts/backfill_chunk_entities.py`)

Builds a single **longest-first alternation regex** over all entity aliases (so the most specific alias wins) plus an **address-core index**, then for each chunk unions matched entity ids into `entity_refs` buckets (`people/llcs/orgs/properties/cases`), sets `entity_ids`, `primary_property_id`, `entity_sides`, and `touches_david`. Existing doc-derived refs are unioned, never overwritten. Supports `--sha-file` scoping for targeted re-linkage (used by the real-time pipeline).

8. Money Graph

The money graph (`scripts/_phase5_money_graph.py` and `_money_*.py`) extracts monetary flows and ties them to properties and parties.

- **Extraction** (`MoneyExtractor`): pulls grounded monetary records (payer, payee, amount, instrument number, date) from chunk text, gated by a fuzzy-match confidence (≥ 80) against source text so amounts are evidence-backed.
 - **Property linkage**: records are linked to canonical properties via `addr_core`.
 - **Reconciliation** (`reconcile`): dedupes/consolidates records on (payer, instrument_no).
 - Output lives in the `money_records` collection and powers the "flow of funds" view and detectors.
-

9. Fraud Detectors, Findings, Timeline & Grounded Facts

9.1 Detectors (`src/detect/`)

`scripts/run_detectors.py` executes a suite of deterministic fraud pattern detectors, each emitting `Finding` documents:

- **Anachronism / backdating** — a document references something that did not yet exist at its stated date.
- **Voidable transfer** — property transfers within look-back windows / for no consideration.
- **Contradiction** — conflicting factual assertions across documents.
- **Instrument conflict** — inconsistent instrument numbers/amounts for the same transaction.
- **Open loop** — an obligation/request with no recorded resolution.
- **LLC timing** — an LLC transacting before its formation date.
- **Insurance changes** — suspicious insurance modifications.
- **Flow detector** (`src/detect/detectors_flow.py`) — money-flow anomalies.

Each `Finding` has a **deterministic_id** (so re-runs upsert rather than duplicate) and a status-preserving upsert (human confirm/reject decisions survive re-runs). Findings carry severity (`critical/high/medium/info`), type, detail, confidence, verbatim evidence quotes with `doc_id`, and linked `property_id`.

9.2 Timeline & events (`src/timeline/builder.py`, `scripts/build_events.py`)

Events are built into an `events` store with **bitemporal** semantics (`src/graph/bitemporal.py`): both when something happened and when it was recorded, enabling accurate ownership intervals (`ownership_intervals`) and point-in-time reconstruction. `timeline_for`, `flow_of_funds`, `evidence_packet`, and `property_graph` (all in `src/timeline/builder.py`) power the Property Detail views and the agent's timeline tools.

9.3 Grounded facts (`src/extract/grounded_facts.py`)

Extracts structured, **verbatim-grounded** facts (each with a `source_quote` and `doc_id`) that feed the property dossier and the portfolio grid's fact counts. Facts are scoped current-vs-historical so the UI shows current encumbrances rather than alarming cumulative totals.

9.4 Authority & edges

`scripts/stamp_authority.py` assigns `doc_authority_score` by a legal authority hierarchy (court order > deed/mortgage > lien > title > insurance > contract), with `DEFAULT_AUTHORITY` fallback.

`scripts/build_graph_edges.py` materializes the `relationships` collection; `scripts/consolidate_properties.py` merges property duplicates by `addr_core`.

9.5 Materialized views

`scripts/build_dossier.py` → `property_dossier`; `scripts/build_dashboard.py` → `dashboard_stats` and `portfolio_grid_cache`. These are read directly by the instant REST dashboards (no live LLM).

10. Retrieval — RAG v2 (every technique)

The RAG v2 pipeline (`src/rag/v2/orchestrator.py`, `hybrid_search.py`, etc.) is the retrieval engine. It is feature-flagged (`rag_v2_enabled`); on any failure the retriever falls back to the legacy v1 flow and records a degrade reason surfaced to the user as a "Retrieval note."

Core infra constants: embeddings `voyage-4-large` (1024-d cosine); Atlas vector index `email_chunks_v2_vector`; BM25 text index `tx_chunks_v2_body_filename_subject`; reranker `rerank-2.5`; query-rewrite LLM `claude-sonnet-4-6`; LLM-reranker `claude-opus-4-8`.

10.1 Hybrid search — five channels fused with RRF

`HybridSearcher` runs five retrieval channels, each producing a ranked list:

Channel	Mechanism	Default top-k
Vector	Atlas <code>\$vectorSearch</code> on embedding. <code>numCandidates = max(150, k×5)</code>	150
BM25/keyword	Mongo <code>\$text</code> sorted by <code>textScore</code>	100
Phrase BM25	quoted <code>\$text</code> (preserves punctuation)	80
Body substring (regex)	escaped <code>\$regex</code> on body/text (deterministic net for money / case# / docket# that <code>\$text</code> tokenization strips)	80
Filename	<code>\$regex</code> on filename OR <code>occurrences.filename</code>	50

With multi-query/HyDE there is **one vector channel per query vector**. The BM25 text index carries field weights (`filename:5, subject:3, body:1`).

Fusion — Reciprocal Rank Fusion (RRF), $\text{score}(d) = \sum 1/(k + \text{rank}_i(d))$ with `k = 60`; deduped on `_id`; truncated to `rrf_fused_cap = 200`. No per-channel weighting — channels are balanced via candidate counts and later multiplicative rescoring.

10.2 Query understanding (deterministic, no LLM)

`query_understanding.extract_signals()` uses pure regex (<1 ms, never raises) to extract: money amounts (with context gating for bare comma-numbers), dates (5 pattern families + year/year-range → `date_from/date_to`), filenames, quoted strings, title-cased phrases, emails, case numbers, docket numbers. It classifies **intent** (`compare/timeline/lookup/summary/opinion`), **complexity** (`is_complex, is_comprehensive`), a `prefer_creation_date` flag (`drafted/signed/filed` verbs), and `keyword_boost_terms` for exact-match rescoring.

10.3 HyDE + multi-query rewriting (LLM)

`QueryRewriter.rewrite()` (`claude-sonnet-4-6`, one call) produces a **HyDE** hypothetical answer (embedded alongside the query) and 2–3 **alternate phrasings** (keyword / conceptual / source-focused). Strict JSON, fully fail-safe (returns original-only on error). All query forms are embedded in one batched Voyage pass.

10.4 Query decomposition (deterministic)

`src/rag/query_decomp.py` splits compound questions on explicit enumerations, multiple `?`, or conjunctions (capped 6 parts). Exposed to the agent as `decompose_search` and used by the finalize sufficiency gate.

10.5 Re-scoring & diversification

`temporal.rescore()` computes `final = RRF × recency × authority × exact_match`:

- **Recency:** exponential decay, 365-day half-life, mapped to `[0.85, 1.20]` (uses `latest_date`).
- **Authority:** filename-pattern tiers — draft/redline `0.90` (demote), order/opinion/judgment `1.20`, stipulation/settlement/9019 `1.12`, agreement/deed/escrow `1.08`, default `1.00`; honors a stamped `doc_authority_score` floor (clamp `1.30`).
- **Exact-match:** +15% per keyword hit (cap `1.5×`).

`diversify()` caps chunks per source cluster (`max_per_cluster=5`). `temporal_diversify()` (compare/timeline intents) round-robins per-year for time spread.

10.6 Expansion passes

- **Full-doc mode**: when the query names a file, pulls whole documents (`full_doc_token_budget=50_000`, `max_docs=4`).
- **Parent-document expansion**: when ≥ 2 hits share an `attachment_id`, pulls the remaining chunks in order (adaptive per-parent budget; `max_parents=5`).
- **Neighbor expansion** (on by default): for single hits, pulls `chunk_index  $\pm$  1` from the same parent — closes the "fact split across a chunk boundary" gap.

10.7 Reranking

- **Voyage** `rerank-2.5` on the top `max(rerank_k $\times$ 3, 30)` candidates; on the deterministic token-limit error it recursively half-splits and merges so nothing is dropped; on any failure it keeps pre-rerank order. `rerank_k` is set by **adaptive-K** (simple/complex/comprehensive $\rightarrow$ 50/70/80).
- **LLM-as-reranker (optional)** `claude-opus-4-8`, `top_n=50`, effort high: scores each passage 0–10 for how directly it answers, preferring operative/recorded instruments with correct party/property/date; reorders the head, never drops passages.

10.8 Ordering & evidence cap

`interleave_for_attention()` places the strongest chunks at the **start and end** of the context (mitigating "lost in the middle"), then `_cap_by_tokens(total_evidence_cap_tokens)` trims overflow **from the middle**, not the tail.

10.9 Orchestrator flow (end to end)

`extract_signals` $\rightarrow$ date-filter merge (`occurrences.date` vs `date`) $\rightarrow$ HyDE/multi-query rewrite $\rightarrow$ embed all query forms $\rightarrow$ 5-channel hybrid search $\rightarrow$ rescore + diversify (+ temporal) $\rightarrow$ adaptive-K $\rightarrow$ Voyage rerank $\rightarrow$ optional LLM rerank $\rightarrow$ full-doc/parent/neighbor expansion $\rightarrow$ interleave $\rightarrow$ hard token cap $\rightarrow$ `RetrievedChunk[]`. Every stage is try/except-guarded; a total collapse returns `[]` and the caller falls back to v1.

11. Agentic Investigator — RAG v3

The production answer path is the **agent** (`src/rag/v3/agent.py`, `AgentRunner.run()`), a Claude tool-use planner that investigates iteratively rather than answering one-shot.

11.1 The loop

1. **SEED**: one `retriever.retrieve(query)` seeds a scratchpad (chunks rendered near-full-body, prompt-cached).
2. **PLAN/ACT/OBSERVE**: a streaming Opus planner (`rag_v3_agent_model`, default `claude-opus-4-8`, adaptive-thinking `effort=xhigh`, `tool_choice=auto`, ephemeral prompt caching) repeatedly picks tools, observes results appended to the scratchpad (with stable `[#N]` indices), and plans the next step.
3. **Sufficiency gate**: the first `submit_final_answer` is intercepted once with a recall self-check before acceptance.
4. **VERIFY + retry**: `_finalize` runs the citation verifier; on failure it does one re-extraction pass.
5. **Hardening** (flag-gated): defense-counsel critic, cross-critic, entailment/coverage/injection checks.

11.2 Budget & termination

`BudgetTracker` enforces `agent_max_tool_calls=30`, `agent_max_total_tokens` (very high ceiling), and `agent_max_wall_clock_s=1200`. On exhaustion it forces finalization via an LLM call (`_force_finalize_via_llm`, 64K output) or a stub.

11.3 The tool palette (`src/rag/v3/tools.py`)

The agent can call (each wrapping a real retrieval/graph/timeline function):

Tool	Wraps / does
<code>search</code>	full RAG v2 pipeline (<code>V2Pipeline.retrieve</code>)
<code>search_by_filename</code>	filename-scoped retrieval
<code>search_timeframe</code>	date-bounded retrieval
<code>fetch_full_document</code>	pulls an entire document's chunks in order
<code>find_quote</code>	locates a verbatim string in the corpus
<code>find_latest_version / compare_versions</code>	version tracking across drafts
<code>verify_claim</code>	runs the deterministic verifier mid-investigation
<code>search_entity_cluster</code>	entity fan-out (<code>graph/fanout.fan_out_chunks</code>)
<code>graph_query, list_documents_for_entity</code>	knowledge-graph lookups
<code>timeline, flow_of_funds, evidence_packet</code>	<code>src/timeline/builder.py</code>
<code>decompose_search</code>	deterministic query decomposition
<code>submit_final_answer</code>	terminal — emits the structured answer + facts

Every internal tool retrieval honors the clean-mode `base_filter`, so privilege exclusion is enforced on all sub-searches, not just the seed.

11.4 Output & trace

The final answer is a forensic memo plus a `facts[]` array (each `{id, claim, source_chunk_id, verbatim_quote, confidence, note}`, `verbatim_quote` min-length enforced). The full reasoning trace (plan, tool calls, chunk refs, verdicts) is persisted to `agent_trace_log` and streamed to the UI's reasoning panel.

12. Verification, Provenance & Evidence Schema

12.1 Deterministic citation verifier (`src/rag/v2/verifier.py`)

For each fact, `verify_facts` checks that the `verbatim_quote` actually appears in cited chunk #N. Verdicts: `VERIFIED`, `UNVERIFIED`, `CITATION_INVALID`.

- **Citation sanity:** non-integer or out-of-range `source_chunk_id` → `CITATION_INVALID`.
- **OCR-tolerant normalization** (`_normalize`): NFKC, curly-quote/dash/NBSP mapping, hyphenation-across-linebreak healing, OCR-jitter de-spacing, split-month healing, \$ 450,000 → \$450,000, intra-number space removal, whitespace collapse + lowercase.
- **GATE 1 — critical tokens:** currency, dates, percentages, big numbers, and years must appear (normalized) regardless of fuzzy score — this catches \$405,000 fuzzy-matching \$450,000. Currency has a formatting-tolerant fallback via `normalize_money/money_matches`.
- **GATE 2 — fuzzy:** `rapidfuzz.fuzz.partial_ratio ≥ 85` → `VERIFIED`.

On a failed fact the agent/pipeline does **one** re-extraction retry; if it still fails, policy is `KEPT_ORIGINAL` (ship the original with an amber verdict rather than fabricate). Outcomes: `VERIFIED_FIRST_PASS`, `VERIFIED_AFTER_RETRY`, `KEPT_ORIGINAL`, `NO_FACTS`, `FALLBACK`. Verification records go to `verification_log`; agent traces to `agent_trace_log`.

12.2 Provenance & clean mode (`src/rag/provenance.py`)

`provenance_footer()` produces structured metadata (counts by corpus/source-type, privileged-source count, date span, verified/unverified fact counts, low-OCR count, and a `clean_mode_leak` flag). Clean mode (`clean_mode_filter()` → `{privilege_status: {$ne: "privileged"}}`) is merged into the seed retrieval **and** passed as `base_filter` to every agent tool; `privilege_status` and `corpus` are declared Atlas vector-index filter paths so exclusion happens at the vector layer. `is_clean_safe` treats missing privilege as privileged (fail-closed). Clean-mode turns are isolated from reusable history.

12.3 Value normalization (`src/rag/normalize_values.py`)

`normalize_money` (with k/m/b multipliers), `all_money`, `money_matches(rel_tol, abs_tol)`, `dates_match`, and `normalize_date_iso` provide the tolerant equality used by the verifier's currency gate and the flow-of-funds/contradiction tooling.

13. Backend API & Server

13.1 Server (server.py)

A FastAPI app ("Legal Advisor RAG API", v1.0.0). CORS allows localhost dev ports (regex) and the production host. A lazy `SessionStore(get_mongo())` backs chat persistence. Entry point: `uvicorn server:app` on `0.0.0.0:8000`.

Method	Path	Auth	Purpose
POST	<code>/api/auth/login</code>	form	Returns JWT
GET	<code>/api/auth/me</code>	JWT	Current user
GET	<code>/api/sessions</code>	JWT	List chats
POST	<code>/api/sessions</code>	JWT	New session
GET	<code>/api/sessions/{id}</code>	JWT	Session + messages
PATCH	<code>/api/sessions/{id}</code>	JWT	Rename (≤ 80 chars)
DELETE	<code>/api/sessions/{id}</code>	JWT	Delete
WS	<code>/ws/chat</code>	JWT (first msg)	Streaming chat
GET	<code>/api/health</code>	none	Status

13.2 Authentication (api/auth.py)

JWT (HS256, `python-jose`), bcrypt password hashing (`passlib`). Single env-configurable user (`AUTH_EMAIL/PASSWORD/NAME`); token TTL default 8 hours (`AUTH_SESSION_MINUTES`). `SECRET_KEY` from `JWT_SECRET_KEY` (with a hardcoded fallback — set the env var in production). Login uses `OAuth2PasswordRequestForm` (URL-encoded username/password).

13.3 WebSocket chat (api/websocket_chat.py)

Client frames: `question` (with `mode: analysis|clean`), `interrupt`, `ping`. Server frames (all carry `session_id`): `start`, `agent_plan/step/done/*`, `token`, `sources`, `verification`, `agent_trace`, `done`, `error`, `pong`. Concurrency: a non-blocking receive loop dispatches each question to a background task; **per-session** chat object (isolated history) + **per-session** lock (serialize within a session, parallel across sessions). Answers persist to DB even if the socket drops (reload-safe). The answer is fully computed (whole agent loop) then **pseudo-streamed** word-by-word (~6 chars, `sleep(0.012)`) for UX. `sources` frames include per-chunk bodies (truncated 8000 chars) with attached verdicts; `agent_trace` is trimmed before sending.

13.4 RAG singleton (api/rag_singleton.py)

Module-level lazy singletons reused across all connections: settings, mongo, embedder, reranker, retriever, anthropic client, and the v2 pipeline. `make_chat()` returns a **fresh** `LegalAdvisorChat` **per session** (own history), wiring summary memory, structured output, verifier (+ `verification_log`), and the agent (+ `agent_trace_log`) based on the config flags. The v2 pipeline is built only if `rag_v2_enabled`; the retriever transparently routes v2 with v1 fallback.

13.5 Sprint-8 REST views (api/views.py)

All JWT-protected, all reading **pre-materialized** collections (instant, no live LLM except `/api/portfolio/cell`):

Path	Purpose
<code>/api/dashboard/stats</code>	Portfolio KPIs
<code>/api/portfolio/properties</code>	Grid rows (side/is_david/litigation/q filters, scoped fact counts)
<code>/api/properties/{id}</code>	Full dossier bundle (title chain, encumbrances, ownership intervals, timeline, flow of funds, findings)
<code>/api/properties/{id}/graph</code>	Property graph payload
<code>/api/properties/{id}/evidence-packet</code>	Evidence packet

Path	Purpose
/api/documents/{id}	Metadata + full OCR transcript
/api/documents/{id}/file	Streams original bytes inline
/api/findings	Findings + facets (severity/type/status)
/api/brief	Daily brief (arrivals, deadlines, open loops, suggested questions)
/api/portfolio/cell (POST)	Ad-hoc grid-cell LLM answer (cached in portfolio_grid_cache)
/api/findings/{id} (PATCH)	Confirm/reject a finding

Original-file resolution tries GridFS `attachment_files` by sha → `attachments_v2` `gridfs_id` → on-disk bases (F:\Title reports, F:/, E:/).

14. Frontend Application

Stack: React 19 + Vite + Ant Design + react-router + axios + react-markdown/remark-gfm + jsPDF. All data flows through the REST/WS API (`frontend/src/api.js`); base URLs from `VITE_API_URL` / `VITE_WS_URL`. Axios auto-injects the bearer token.

14.1 Routing & shell (`App.jsx`)

On mount, validates the stored token via `getMe()`. Unauthenticated → `<Login>`. Authenticated → routes: `/chat` (full-screen), and a shell-wrapped group `/portfolio`, `/brief`, `/findings`, `/how-to-ask`, `/properties/:id`. Ant theme primary `#234a52` (teal), Inter font.

14.2 Chat (`Chat.jsx`)

The streaming investigation UI. Per-session UI state keyed by `session_id` (never shared). WebSocket via `useWebSocket` (StrictMode-safe, reconnect-safe). Handles all agent frames to drive a live reasoning panel. Optimistic user-message append + streaming indicator ("`[search]` Retrieving evidence..." → "`[writing]` Claude is writing..."). Clean-mode toggle. Interrupt/Stop button. History replay rehydrates sources + verification + trace so citation chips and the evidence drawer work on saved chats.

14.3 Citations & evidence

`renderWithCitations` regex-splits answer text on `[#N]` / `[#fN]` into `CitationChips`, color-coded by verdict (green VERIFIED / amber UNVERIFIED / red CITATION_INVALID / teal neutral), with hover quote previews; clicking opens the **EvidenceDrawer** at that source. `Sources` renders collapsible source cards (cited-first, rerank score, "supports N verified claims"). `EvidenceDrawer` shows metadata, a jump-to-source picker, per-claim verdict cards, and full source text with matched spans highlighted. `VerificationBanner` summarizes the outcome.

14.4 Agent reasoning panel (`AgentReasoningPanel.jsx`)

Renders the live investigative trace: status, a tool-calls progress meter, a live wall-clock timer, a Stop button, tool icons/labels for every agent tool, and expandable step rows showing tool input, new chunk refs, and errors.

14.5 Workspace pages

- **PortfolioGrid**: KPI stat cards + an Ant Table of properties (side, owners, title count, insurance, equity/mortgage, foreclosure, litigation, scoped fact counts). Ad-hoc **" + Column"** runs `portfolioCell` across all rows (4 concurrent workers, server-cached).
- **PropertyDetail**: tabs for Property map (graph), Overview, Title reports (+ chain of title), Encumbrances, Insurance, Timeline, Flow of funds, Findings, Documents (chain of custody). Exports an **Evidence Packet** as JSON and as a full court-ready **PDF** (identification, financials, ownership history, title chain, encumbrances, insurance, timeline, flow of funds, findings, numbered chain-of-custody references with SHA-256).
- **DocumentViewer**: Original/Transcript(OCR) toggle; embeds PDFs/images or offers download; shows `date/vendor/pages/sha256`.
- **FindingsDashboard**: severity-filtered table with a detail drawer (evidence quotes, confidence) and confirm/reject review.
- **BriefDashboard**: "Insight Engine" cards — suggested next questions, approaching deadlines (color-coded), open loops, new arrivals.
- **Sidebar**: chat search, inline rename, delete-with-confirm, relative timestamps.

14.6 Answer PDF export (`exportAnswerPdf.js`)

Builds an A4 jsPDF of the answer with Markdown structure preserved, then a numbered "References & Sources" section mapping each `[#N]` to its source title/date/from + verbatim quotes (Latin-1 transliteration for jsPDF's built-in fonts).

15. Database Layer & Collection Catalog

15.1 MongoClientWrapper (src/db/mongo.py)

Builds MongoClient(tz_aware=True, uuidRepresentation="standard", appname="fraud-emails-ingestor") and exposes named handles: emails, attachments, folders, runs (ingestion_runs), errors (ingestion_errors), chunks (email_chunks), and a GridFS bucket attachment_files. ensure_indexes creates all indexes including the emails text index tx_subject_body. EmailRepository (repository.py) provides the write API (run lifecycle, folder upsert, dedup lookups, bulk email upsert, attachment→GridFS storage, linkage).

15.2 Collection catalog

Collection	Written by	Purpose
emails	ingestion	canonical email documents
attachments	ingestion	attachment metadata (binary in GridFS)
attachment_files.*	GridFS	attachment binaries
attachments_v2	OCR	one OCR'd row per attachment (dedup by sha)
email_chunks	legacy v1 chunker	v1 chunks
email_chunks_v2	build_email_chunks_v2	production vector chunks (+ enrichment)
entities	entity resolution	people/lcs/orgs/properties/cases
entity_review	resolution	grey-zone human review queue
relationships	build_graph_edges	graph edges
money_records	money graph	monetary flows
events	build_events	bitemporal event store
findings	run_detectors	fraud findings
property_dossier	build_dossier	per-property materialized bundle
dashboard_stats	build_dashboard	portfolio KPIs
portfolio_grid_cache	views/build	cached grid-cell answers
fact_clusters	grounded facts	clustered facts
documents	doc ingestion	non-email documents
folders	ingestion	folder tree
ingestion_runs / ingestion_errors	ingestion	run + error tracking
gmail_watch_state	gmail_watch	push watch state
chat_sessions	API	chats + messages
verification_log	verifier	fact verification records
agent_trace_log	agent	agent reasoning traces
eval_results	eval harness	regression/eval scores

16. Configuration Reference

All configuration is centralized in `config/settings.py` (`Settings.load()`), loaded from `.env` (`override=True`). The dataclass is frozen; `settings` is `None` unless `MONGO_URI` is set. Key groups (defaults shown):

Core / infra: `mongo_db_name=fraud_emails`, `batch_size=100`, `max_body_chars=2,000,000`, `attachment_max_bytes=50 MB`.

RAG core: `embedding_model=voyage-4-large`, `embedding_dim=1024`, `rerank_model=rerank-2.5`, `claude_model=claude-opus-4-8`, `claude_max_output_tokens=40960`, `chunk_size_tokens=1000`, `chunk_overlap_tokens=200`, `retrieval_top_k=50`, `rerank_top_k=8`.

OCR: `ocr_dpi=300`, `ocr_text_layer_min_chars=80`, `ocr_vision_enabled=true`, `ocr_vision_model=claude-sonnet-4-6`, `ocr_vision_min_pages=3`, `ocr_vision_dpi=180`, `ocr_vision_max_concurrency=8`, `ocr_vision_budget_usd=15.0`.

RAG v2 feature flags (master `rag_v2_enabled`; Sprint 1 `hybrid_search/filename_lookup/hyde/multi_query/date_filters/enhanced_prompt`; Sprint 2 `parent_doc/temporal_diversity/adaptive_k/rescoring/summary_memory`; Sprint 2.5 `full_doc_mode/interleaved_ordering/xml_sources`).

Sprint 3 verified answers: `rag_v2_structured_output`, `rag_v2_citation_verifier`, `rag_v2_verifier_retry`, `rag_v2_verifier_threshold=85.0`, `rag_v2_verifier_log`.

Sprint 4 agent: `rag_v3_agent_enabled`, `rag_v3_agent_max_tool_calls=30`, `rag_v3_agent_max_total_tokens=15,000,000`, `rag_v3_agent_max_wall_clock_s=1200`, `rag_v3_agent_model=claude-opus-4-8`, `rag_v3_agent_max_tokens_per_call=64000`, `rag_v3_agent_seed_with_initial_search=true`, `rag_v3_agent_trace_log=true`, `rag_v3_agent_effort=xhigh`.

v2 model/channel tunables: `rag_v2_query_rewriter_model=claude-sonnet-4-6`, `rag_v2_llm_reranker=true` (`claude-opus-4-8`, `top_n=50`, `effort high`), `rrf_k=60`, `rrf_fused_cap=200`, `vector_top_k=150`, `bm25_top_k=100`, `phrase_top_k=80`, `body_regex_top_k=80`, `filename_top_k=50`, `adaptive-K 70/100/120`, `parent-doc 8000/5/20`, `full-doc 50000/4`, `total_evidence_cap_tokens=500000`, corpus targets `email_chunks_v2` + `email_chunks_v2_vector`.

17. Real-Time Automation & Deployment

The event-driven ingestion runs on a DigitalOcean server via systemd (`deploy/`):

- `gmail-push-worker.service` — the always-on Pub/Sub subscriber.
- `gmail-watch.service` + `gmail-watch.timer` — daily re-arming of the Gmail watch (which expires ~7 days).

Google Cloud Pub/Sub topic `projects/mango-500409/topics/gmail-boris` and subscription `gmail-boris-sub` deliver notifications. `deploy/REALTIME_INGEST_SETUP.md` documents the GCP + server setup, and the credentials (`client_secret.json`, `gmail_token.json`, `service-account JSON`) are git-ignored.

Two server profiles exist: the full `requirements.txt` (workstation / data-prep, includes OCR/PST libs) and the slim `requirements_server.txt` (read-only chat backend — excludes OCR). See the §3.4 caveat: heavy enrichment must run where the OCR stack and RAM exist.

18. Appendix A — Constants & Tunables Quick Reference

Constant	Value	Where
Embedding model / dim	voyage-4-large / 1024 (cosine)	settings, embedder
Reranker	rerank-2.5	reranker
Agent / generation model	claude-opus-4-8	settings, agent
Rewrite / summary / OCR model	claude-sonnet-4-6	settings, rewriter, OCR
LLM reranker	claude-opus-4-8, top_n 50, effort high	orchestrator
Chunk size / overlap	1000 / 200 tokens	chunker, settings
Tokenizer	tiktoken cl100k_base	tokens.py
Force-vision sentinel	ocr_min_chars = 10,000,000	ocr_attachments_v2
OCR cascade	Claude Sonnet → GPT-5 → RapidOCR (12-pg cap)	extractor
OCR budget	\$15 (_SpendGuard)	claude_ocr
RRF k / fused cap	60 / 200	orchestrator
Vector / BM25 / phrase / regex / filename top-k	150 / 100 / 80 / 80 / 50	orchestrator
Recency window / half-life	[0.85, 1.20] / 365 d	temporal
Authority tiers	0.90 / 1.00 / 1.08 / 1.12 / 1.20 (clamp 1.30)	temporal
Exact-match boost cap	1.5×	temporal
Adaptive-K (simple/complex/comprehensive)	50/70/80 (v2) · 70/100/120 (settings)	orchestrator/settings
Neighbor expand window	±1 (max 40)	parent_doc
Evidence cap	100K (v2 default) · 500K (settings) tokens	orchestrator/settings
Verifier fuzzy threshold / min quote	85.0 / 6 chars	verifier
Agent budgets	30 calls / 1200 s	settings

19. Appendix B — Known Configuration-Drift Notes

These are real inconsistencies surfaced during the code review, listed so maintainers can reconcile them:

1. **Model-name display drift:** config defaults point at `claude-opus-4-8`, but the `/api/health` endpoint, Chat top-bar, and Login footer display "Claude Sonnet 4.6."
2. **Evidence-cap divergence:** `orchestrator.py`'s internal `V2Settings` default is `total_evidence_cap_tokens = 100_000`, while `config/settings.py` default is `500_000` (the settings value wins when wired through `rag_singleton`).
3. **Adaptive-K divergence:** `V2Settings` defaults 50/70/80 vs `settings.py` 70/100/120.
4. **Two WS/REST auth allow-lists:** `api/auth.py` uses an env-configurable single user; `api/websocket_chat.py` has a hardcoded allow-list `{rakeshsir@mtreh.com}` — changing `AUTH_EMAIL` authenticates REST but would be rejected by the WS.
5. **JWT secret fallback:** `api/auth.py` has a hardcoded `SECRET_KEY` fallback — production must set `JWT_SECRET_KEY`.
6. `extract_from_bytes` **method labelling:** a PDF transcribed entirely by GPT-5 (`openai_vision`) can be labelled `pdf_text` at the top level even though per-page `method` is correct.
7. **Slim-server OCR gap (operational):** `requirements_server.txt` omits OCR libs, so the real-time worker's OCR step fails on the 1 GB droplet — capture succeeds, enrichment does not, until a workstation backfill runs.
8. `query_expansion.py` exists but is not wired into the v3 agent path.

End of document. Every technique above is grounded in the actual source of the Mango Tree Legal Evidence Engine codebase.